Text

IMDb

LECTURE 17 | GÉRON CH. 14

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

A new kind of input

Sixteen lectures on numbers, pixels and ordered numbers. Today the input is **free text**: variable length, discrete, and with no natural numeric encoding at all.

- A review is a sequence of *symbols*, not of measurements
- There is no distance between two words that anything gave us
- Sequence machinery from Lecture 16 still applies — once we decide what a step *is*

> **TWO QUESTIONS, AND THIS LECTURE IS BOTH**
>
> **How do symbols become vectors?** — tokenisation and embeddings. And **what loss does a classifier over K symbols minimise?** — which is today's derivation, and it is the loss every classifier in this course has quietly been using since Lecture 4.

Today

1. The task, the corpus, and the baselines that a bag of words already gives you (15 min)
2. **The mathematics** — softmax, cross-entropy and logits (25 min)
3. **The method** — subword tokenisation, trainable embeddings, and a recurrent classifier (35 min)
4. The same architecture with pretrained embeddings (15 min)

PART ONE

The brief

15 minutes · the problem, the data, the stakeholder

The stakeholder

A streaming service's customer-feedback desk.

Every title carries free-text reviews. Nobody reads them. The desk wants, for each new review, a single label — *satisfied* or *not* — so that the unhappy ones can be routed to a human within the hour.

- No star rating is available at the point where they need the decision
- The volume is far past what a person can triage
- They are prepared to accept mistakes; they are not prepared to accept silence

What they actually asked for

Their words	What that specifies
“tell us if it is a complaint”	binary classification
“from the text alone”	one variable-length string in, one label out
“within the hour”	inference cost matters; training cost does not
“we will read the ones you flag”	the errors are absorbed by a human

Four sentences, and every one of them is a design decision you would otherwise have made silently.

Free text has no columns

Everything in Chapters 1–13 arrived as a table: a fixed set of features, in a fixed order, each a number or a category.

> **A REVIEW IS NONE OF THOSE**
>
> Variable length. No fixed positions. A vocabulary that is open — the next review can contain a word that has never been written before.

So before any model, there is a question no previous application had to answer: **what is the unit?**

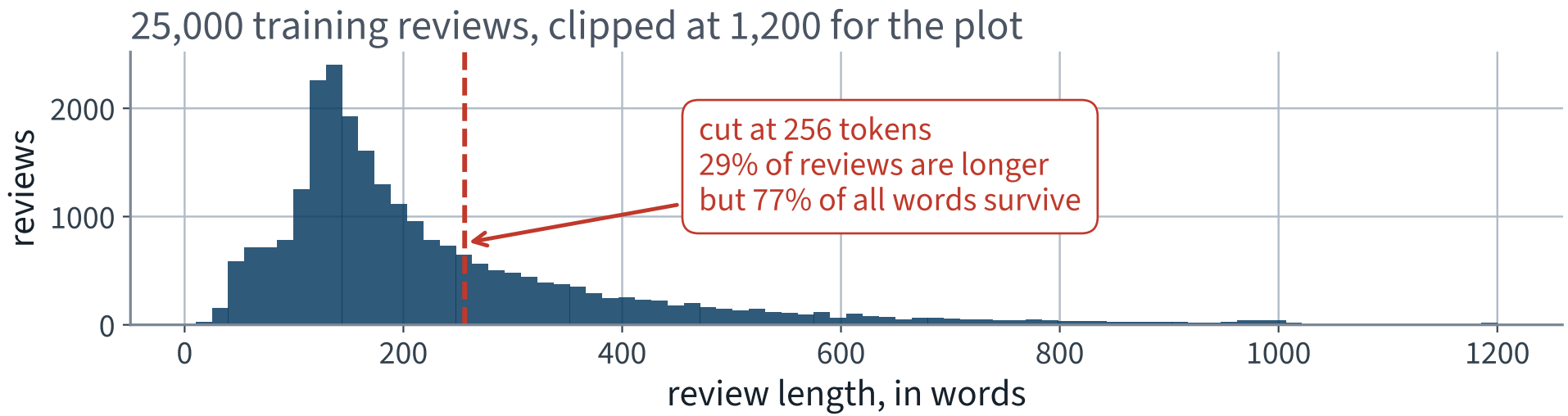
The corpus

The Large Movie Review Dataset — the corpus the book uses for sentiment. Fetched from Stanford directly, about 80 MB.

```
URL = "https://ai.stanford.edu/~amaas/data/sentiment/aclImdb_v1.tar.gz"

def load_imdb():
    if not DATA.is_dir():
        urllib.request.urlretrieve(URL, tarball)
        with tarfile.open(tarball) as t:
            t.extractall(path=CACHE, filter="data")
    ...
```

A function, not a manual download — the same rule as Lecture 1. The split into train and test ships with the corpus, so every course in the world reports on the same reviews.

How long is a review?



A long tail. Wherever you cut, you cut something — so the cut is a decision to make on purpose and record.

The vocabulary is the problem

Lower-case, split on non-letters, count:

Quantity	Value
Distinct words in the training reviews	87,171
Words that appear exactly once	35,344
✗ … as a share of the vocabulary	**✗ 41%**

Two words in five are seen once and never again.

You cannot learn a vector for a word from one example, and you cannot afford a row in a table for every one of them.

What the model has to produce

$$\text{string} \longrightarrow \mathbb{R}^2$$

Two numbers per review: one score per class. Everything between the string and those two numbers is what we build today.

- A **tokenizer**, turning a string into a sequence of integers
- An **embedding**, turning each integer into a vector
- A **recurrent layer**, turning the sequence into one vector
- A **linear head**, turning that vector into two numbers

Only the first two are new. The last two are Lectures 10 and 16 unchanged.

PART TWO

Choose a metric and commit

15 minutes · anchored, then written down

Which metric?

Lecture 3's rule: *count the classes before choosing a metric.* We counted. They are 50.0% to 50.0%.

> **SO ACCURACY IS DEFENSIBLE HERE**
>
> Under a 50/50 base rate, the degenerate classifier scores 50% and has nowhere to hide. That is the condition Lecture 3 said accuracy needs, and this corpus satisfies it.

It is defensible *on this corpus*. Real feedback is not balanced, and the first slide of the next application you build should ask the question again.

Bag of words, precisely

One column per term in the vocabulary; one row per review; the entry is how often the term occurs, weighted down if the term is common everywhere.

$$\text{tf-idf}(t, d) = \underbrace{\text{count}(t, d)}_{\text{term frequency}} \times \underbrace{\log \frac{N}{1 + n_t}}_{\text{inverse document frequency}}$$

N is the number of documents and n_t the number containing t. The second factor is what stops *the* and *movie* dominating the first. Scikit-Learn uses a smoothed variant, $\log \frac{1+N}{1+n_t} + 1$, and normalises each row to unit length.

✗ Note what is not in that formula: position. The bag has no order, so it cannot see the negation two slides ago.

The whole anchor, in eleven lines

```
1  from sklearn.feature_extraction.text import TfidfVectorizer
2  from sklearn.linear_model import LogisticRegression
3
4  # fit on the training reviews ONLY — the vectoriser is a fitted estimator
5  vec = TfidfVectorizer(min_df=2, ngram_range=(1, 2),
6                        max_features=200_000)
7  X_train = vec.fit_transform(train_x)
8
9  clf = LogisticRegression(max_iter=2000, C=4.0).fit(X_train, train_y)
10 acc = (clf.predict(vec.transform(test_x)) == test_y).mean()
11 print(f"tf-idf + logistic regression: {acc:.1%}")
```

Line 7 is `fit_transform` on the training reviews; line 10 is `transform` on the test reviews. Different verbs. Hold on to that — it is the last thing we do in the next lecture.

The anchors, measured

Model	Features	Test accuracy	Scoring 25,000
Always predict one class	—	50.0%	—
tf-idf, single words	44,798	88.5%	1.4 s
✓ tf-idf, words and pairs	200,000	**90.1% ✓**	**3.6 s ✓**

90.1%, from counting words, in 10 seconds.

Adding pairs of adjacent words buys most of a point, and pairs are the cheapest way a bag can see *not good*. The last column is the brief's *other* requirement — “within the hour” — and it is the one the metric cannot see, so it goes on the sheet too.

THE MATHEMATICS

Softmax, cross-entropy and logits

25 minutes · examinable

The question

Your model's last line was

```
return self.head(torch.cat([h[0], h[1]], dim=1))     # (B, 2)
```

Two unbounded real numbers per review. No softmax anywhere. You handed them to `nn.CrossEntropyLoss` and it worked.

> **WHY YOU NEED THE ANSWER**
>
> Because the same two numbers are what every classifier in the rest of this course produces, and because one line that looks like an improvement makes the model measurably worse.

Logits

The output of the last linear layer, before anything is done to it.

$$\mathbf{z} = W\mathbf{h} + \mathbf{b} \in \mathbb{R}^K$$

- Unbounded, positive or negative, and they do not sum to anything
- Only their **differences** carry information — the next five minutes are about that
- The name is historical: for $K = 2$ the difference $z_1 - z_0$ is the log-odds, the logit of Lecture 4

That last point is worth holding on to. Logistic regression is this construction with $K = 2$ and one difference.

Softmax

The map from K unbounded numbers to a probability vector:

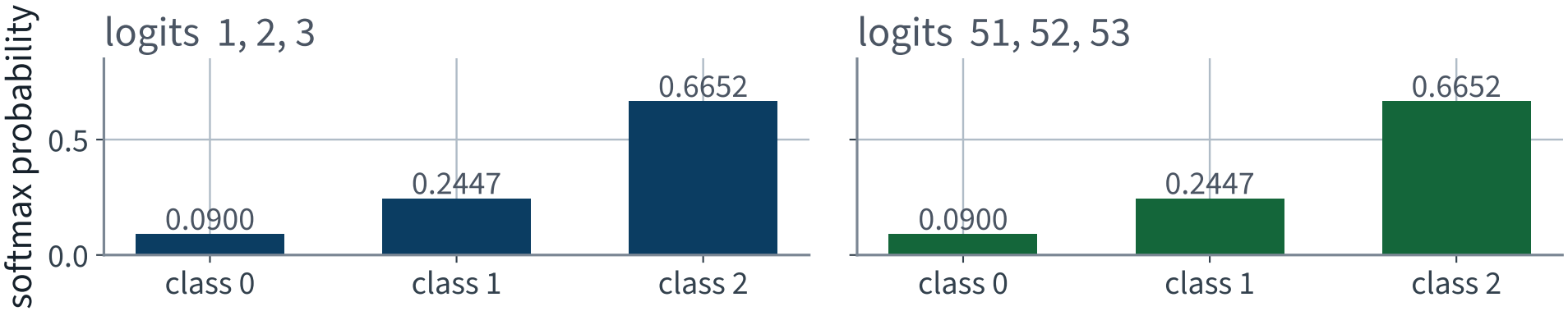
$$\sigma(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}} \qquad k = 1, \ldots, K$$

- Every entry is positive, because $e^x > 0$
- They sum to 1, because the denominator is the sum of the numerators
- It is order-preserving: $z_i > z_j \iff \sigma(\mathbf{z})_i > \sigma(\mathbf{z})_j$

The third point already tells you that `argmax` does not need it. A prediction never requires softmax.

Add fifty to every logit

identical to 6e-08 — adding a constant to every logit changes nothing



The two panels agree to 5.96046e-08. Not approximately: the same numbers.

Why a mathematician's triviality is an engineer's disaster

The *function* ignores the shift. `float32` does not.

$$e^x \text{ overflows float32 when } x > \log(3.4 \times 10^{38}) \approx 88.7$$

- A confident network reaching logits of 100 is completely ordinary
- The exponential of 100 is not representable; the sum becomes `inf`; the ratio becomes `nan`
- And `nan` propagates into every parameter through `backward()`

The standard repair

$$\sigma(\mathbf{z})_k \;=\; \sigma(\mathbf{z} - z_{\max}\mathbf{1})_k \;=\; \frac{e^{z_k - z_{\max}}}{\sum_j e^{z_j - z_{\max}}}$$

- The largest exponent is now exactly 0, so $e^0 = 1$ and nothing overflows
- The smallest may underflow to 0, which is harmless: it was going to be negligible anyway
- The denominator is at least 1, so the division is safe

Shift invariance is not a curiosity about the formula. It is the licence to do this, and without it there would be no numerically safe softmax.

Cross-entropy

Between a target distribution $\mathbf{y}$ and a predicted $\mathbf{p}$:

$$H(\mathbf{y}, \mathbf{p}) = -\sum_{k=1}^{K} y_k \log p_k$$

A hard label is one-hot — $y_c = 1$, the rest 0 — so every term but one dies:

$$L = -\log p_c$$

Reading that loss

p_c	$-\log p_c$	Reading
1.00	0.00	certain and right
0.50	0.69	a coin, on two classes
0.10	2.30	wrong, and not yet confident about it
✗ 0.01	**✗ 4.61**	**✗ wrong and certain — unbounded penalty**

$-\log$ is what makes confident mistakes expensive. A loss linear in p_c would price the last two rows almost the same.

On two balanced classes, an untrained model should show a loss near 0.69. If your first epoch starts far from that, something is wrong before any training has happened.

The gradient with respect to the logits

Substituting softmax into the loss and using $\log \frac{a}{b} = \log a - \log b$:

$$L = -\log \frac{e^{z_c}}{\sum_j e^{z_j}} = -z_c + \log \sum_j e^{z_j}$$

The exponential of the true class has cancelled.

That is not a simplification for the slide — it is the whole reason the combined form is the one implemented. Hold on to it for four slides.

Differentiate it

The first term contributes only when $k = c$:

$$\frac{\partial}{\partial z_k}(-z_c) = -[k = c] = -y_k$$

The second is the log-sum-exp, whose derivative is softmax:

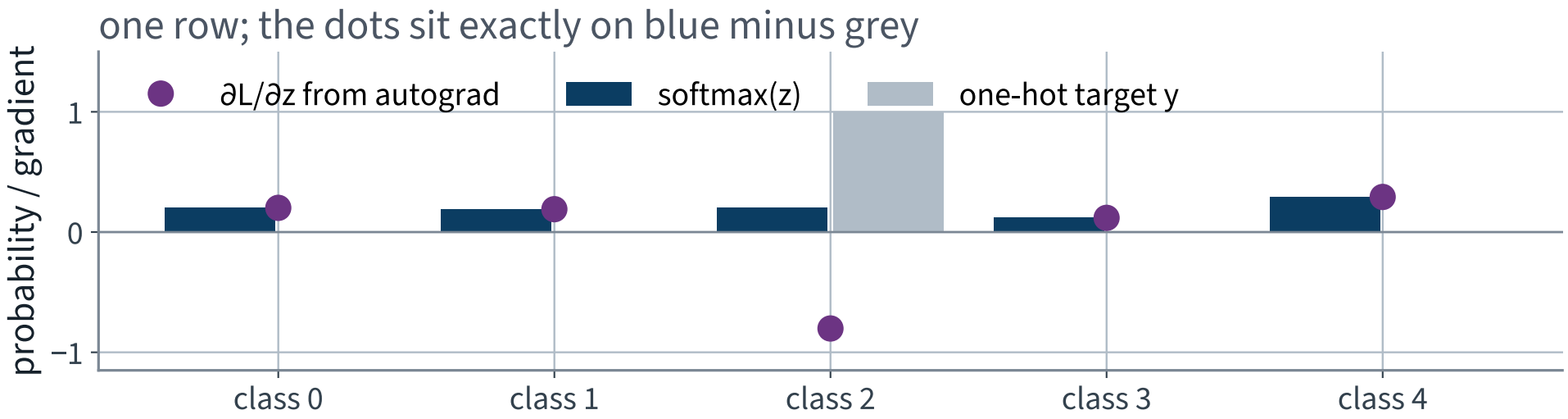
$$\frac{\partial}{\partial z_k} \log \sum_j e^{z_j} = \frac{e^{z_k}}{\sum_j e^{z_j}} = p_k$$

Prediction minus target

- Three lines, no chain rule through the softmax, no Jacobian
- Every component lies in $[-1, 1]$, so the signal into the network is **bounded** however wrong the model is
- The components sum to zero, because both vectors sum to one — the gradient has no component along $\mathbf{1}$, which is exactly the direction softmax could not see
- It vanishes only when the prediction is exactly the target

> Compare with mean squared error on probabilities, whose gradient carries a factor $p_k(1 - p_k)$ — near zero precisely when the model is confidently wrong, which is when you most need a gradient.

One row, checked against autograd



The dots are what PyTorch computed. The bars are what the algebra predicts. They agree to 5.55112e-17.

So why does the loss want logits?

Because of that cancellation.

If it took probabilities

$$L = -\log\left(\frac{e^{z_c}}{\sum_j e^{z_j}}\right)$$

✗ Form every exponential, divide, and then take the logarithm of a number that may be 10^{-40}.

Taking logits

$$L = -z_c + \log\sum_j e^{z_j}$$

✓ One pass, with the maximum subtracted inside the log-sum-exp. The exponential and the logarithm never meet.

And the gradient falls out as $\mathbf{p} - \mathbf{y}$ without differentiating through the division.

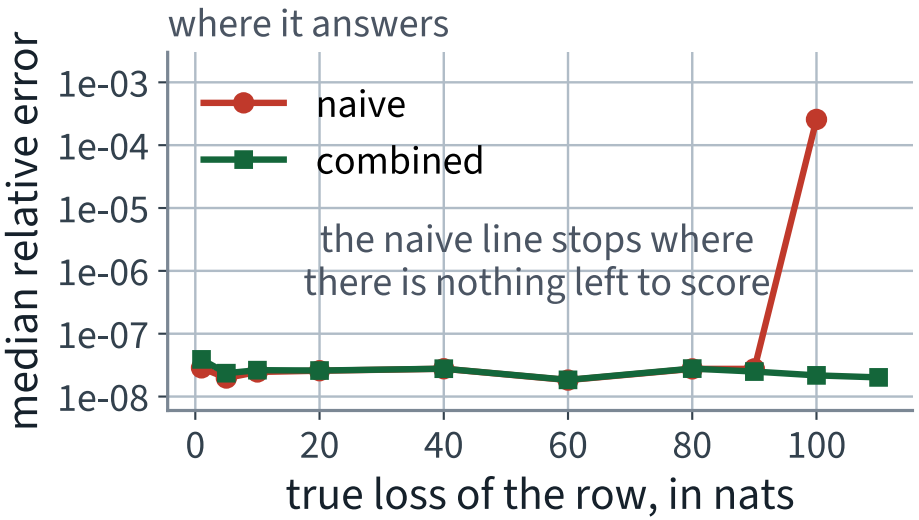
Do not take that on trust — measure it

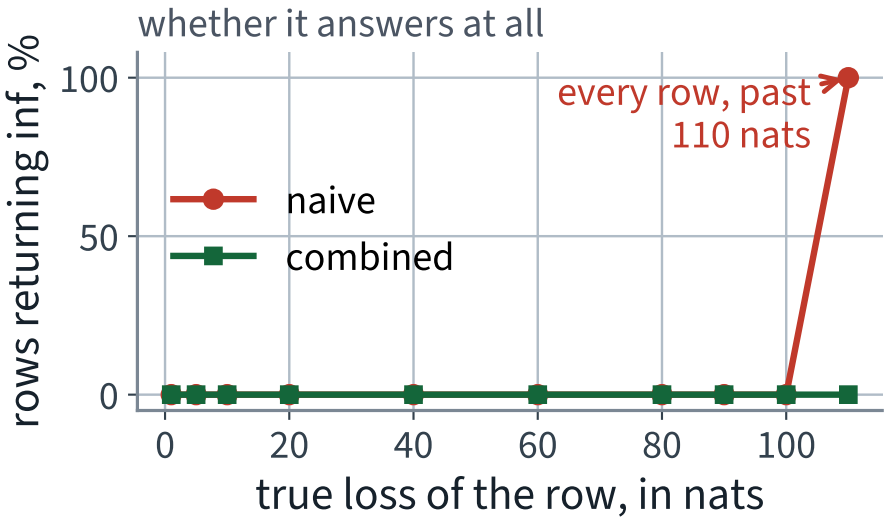
Two ways to compute the same loss in `float32`, scored against a `float64` reference:

```
# naive: probabilities first
p     = np.exp(z) / np.exp(z).sum(1, keepdims=True)
naive = -np.log(p[rows, y])

# stable: the combined form
m      = z.max(1, keepdims=True)
stable = -(z[rows, y] - (m[:, 0] + np.log(np.exp(z - m).sum(1))))
```

2,000 rows of 10 logits at each of ten settings, sweeping **how wrong the row is** — the true class placed 1 to 110 nats below the largest logit. That is the quantity the failure depends on: the naive form has to represent $e^{-\text{loss}}$ as a `float32`.

Where `float32` gives up





✗ **Left: the two agree to `float32` precision until the loss reaches about 90 nats; at 100 the naive form is four orders of magnitude worse. Right: at 110 it returns nothing at all. The combined form does neither.**

And the one that does not announce itself

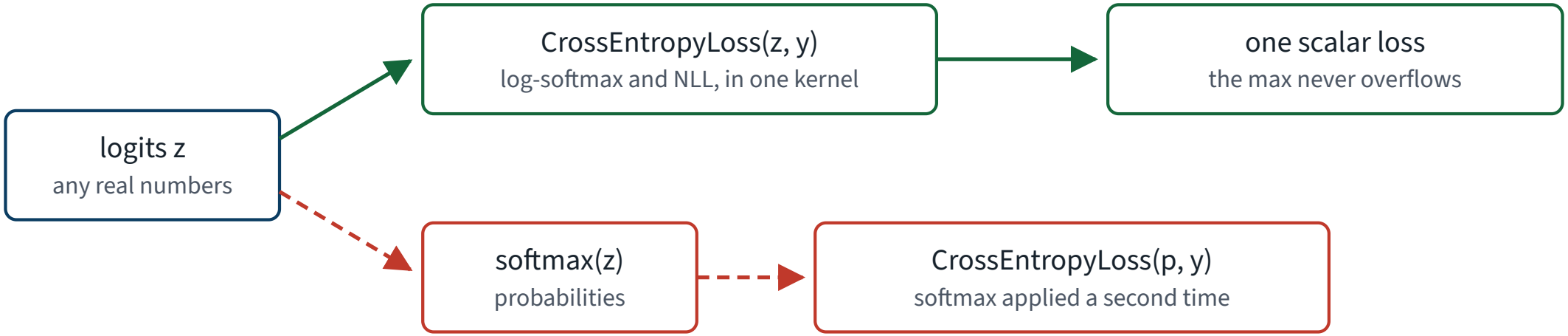
```
z = [0., 0., -100.]   float32,   true class = 2
```

Computed as	Loss	Error
float64 reference	100.69315	—
✗ naive, probabilities first	✗ 100.63987	✗ 0.0533
✓ combined, from logits	100.69315 ✓	1.42861e-06 ✓

✗ Nothing overflowed. The probability $e^{-100}/(2+e^{-100})$ is 1.96182e-44, which `float32` can only hold as a *subnormal* with a handful of significant bits — so the logarithm of it is wrong in the second decimal, finitely, plausibly, and without a warning.

Two paths from the last layer

What the last layer hands to the loss



It runs. It trains. The gradient is squashed toward zero, and nothing raises.

On the green path only: $\partial L / \partial z = \text{softmax}(z) - y$

prediction minus target, with every exponential cancelled before it is ever formed

Both paths run. Only one of them computes the loss you wrote down.

The last piece: what cross-entropy is crossing

Shannon entropy — the average surprise of a draw:

$$H(\mathbf{y}) \;=\; -\sum_{k} y_k \log y_k$$

Kullback–Leibler divergence — how much worse you do coding draws from $\mathbf{y}$ as if they came from $\mathbf{p}$:

$$D_{\mathrm{KL}}(\mathbf{y} \parallel \mathbf{p}) \;=\; \sum_{k} y_k \log \frac{y_k}{p_k} \;\geq\; 0$$

Non-negative, zero only when the two agree, and not symmetric.

They differ by a constant

$$H(\mathbf{y}, \mathbf{p}) \;=\; H(\mathbf{y}) \;+\; D_{\mathrm{KL}}(\mathbf{y} \,\|\, \mathbf{p})$$

Add and subtract $\sum_k y_k \log y_k$ in $-\sum_k y_k \log p_k$. That is the whole proof.

AND FOR A ONE-HOT TARGET

$H(\mathbf{y}) = 0$, so minimising cross-entropy **is** minimising the KL divergence to the label. The two objectives are the same objective.

the derivation, summarised

1. Softmax is invariant under adding a constant to every logit, which is the licence to subtract the maximum before exponentiating
2. Cross-entropy against a one-hot target is $-\log p_c$ — one number, and unbounded when the model is confidently wrong
3. $\partial L / \partial \mathbf{z} = \mathbf{p} - \mathbf{y}$: bounded, summing to zero, and free of the softmax Jacobian
4. The loss consumes logits because $-z_c$ cancels the exponential, and computing it any other way loses digits and then overflows
5. Cross-entropy and KL differ by the entropy of the target, which is zero for a hard label

Point 3 returns in thread 12, where the contrastive objective is a cross-entropy over similarities.

THE METHOD

Tokens, embeddings, and a classifier

35 minutes · examinable

The notebook for this lecture

Open Lecture 17 in Colab

- **Runs on free CPU.** IMDb is subsampled and the model is small
- The softmax shift-invariance and the max trick, checked numerically — including the row where the naive version overflows
- The gradient of cross-entropy with respect to the logits, verified against autograd: it really is prediction minus target

> **WHAT TO CHANGE**
>
> Apply a softmax before a loss that expects logits. Nothing raises, the model still trains, and it trains worse — the double-softmax failure, measured.

The plan, in four boxes

Stage	In	Out
Tokenize	one string	`(256,)` integers
Embed	`(B, 256)`	`(B, 256, 128)`
Recur	`(B, 256, 128)`	`(B, 128)`
Head	`(B, 128)`	`(B, 2)`

Write those four shapes on your sheet. Reviewer question three — *what is the shape here?* — is the question that catches most of what goes wrong in the next hour.

Step 1 · what is a token?

The unit the model counts as one thing.

- **Character** — tiny vocabulary, sequences six times longer, and every fact about a word has to be relearned from letters
- **Word** — the obvious choice, and the one that breaks
- **Subword** — the compromise the field settled on, and the subject of the next ten minutes

This is a modelling decision, not a preprocessing detail. It fixes the vocabulary, the sequence length, and what the model can possibly represent.

Our word tokenizer, in full

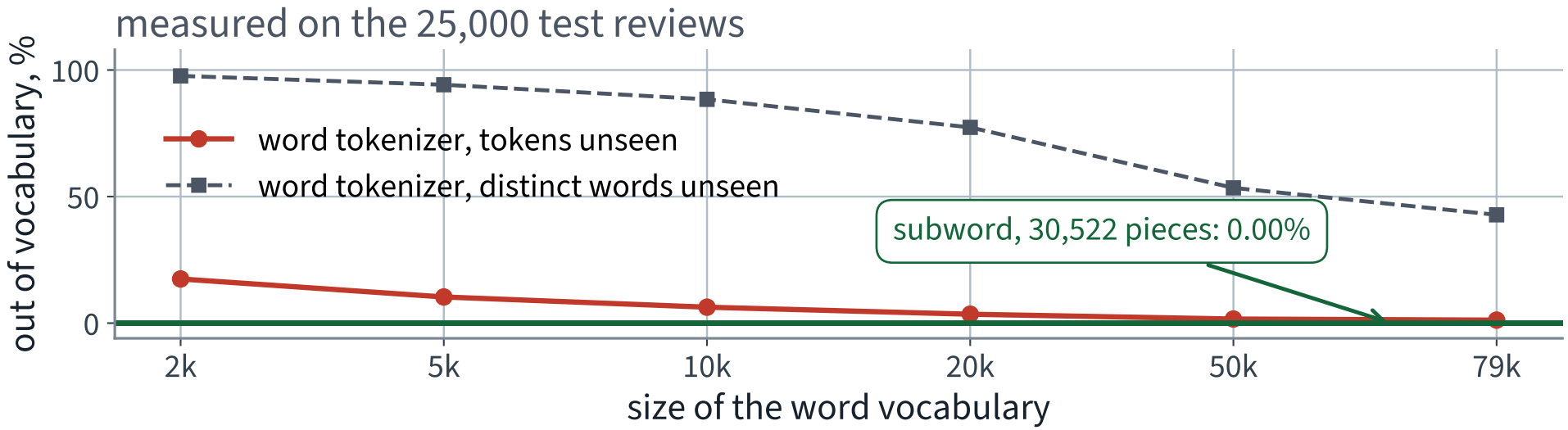
```
WORD_RE = re.compile(r"[a-z0-9']+")

def word_tokens(s):
    return WORD_RE.findall(s.lower().replace("<br />", " "))
```

That is the whole specification. Three decisions are already made in it, and every one is arguable:

- Case is destroyed — *Terrible* and *terrible* become one token
- Punctuation is destroyed — so is the exclamation mark
- The corpus is HTML, and `<br />` is not a word

✗ If you had not looked at the raw text you would have learned `br` as one of the commonest words in the corpus.

What a word vocabulary costs you



✗ The token rate looks survivable. The rate over *distinct* words never does — and those are the informative ones.

Step 1b · subword tokenisation

Keep whole words for the common ones. Break the rest into pieces that *are* in the vocabulary.

- Start from characters, which cover everything by construction
- Repeatedly merge the commonest adjacent pair into a new token, until the vocabulary reaches the size you asked for
- Common words survive whole; rare words end up as two or five pieces

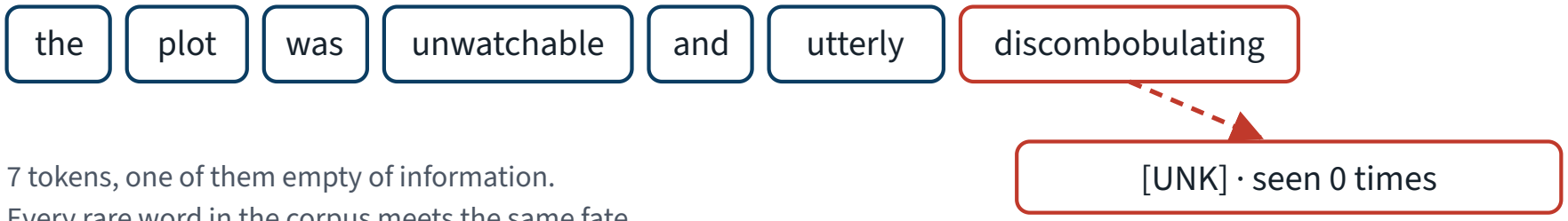
Byte-pair encoding, and WordPiece, which differs only in the criterion for which pair to merge. Chapter 14 names both.

The merges are learned **once**, on some corpus, and then shipped. That is the first pretrained thing we use today.

The same sentence, twice

The plot was unwatchable and utterly discombobulating.

our word tokenizer · 20,000 words, ranked by frequency in the training split



7 tokens, one of them empty of information.
Every rare word in the corpus meets the same fate.

WordPiece · 30,522 pieces, learned once on a corpus we never saw

13 pieces, none unknown. The three pieces of unwatchable keep the negation the word tokenizer threw away, and an unseen word falls back to pieces, a piece to characters. The vocabulary cannot miss.

Above, the rarest and most informative word became nothing. Below, it became four pieces the model has vectors for.

Why that is a structural fix, not a patch

> **THE VOCABULARY CANNOT MISS**
>
> An unseen word falls back to pieces; an unseen piece falls back to characters; the character set is closed. There is no input for which the tokenizer has to produce `[UNK]`.

- Morphology survives: *un* + *watch* + *able* keeps the negation that `[UNK]` destroyed
- Misspellings degrade rather than vanish
- The vocabulary is a fixed size you choose, whatever the corpus

Step 2 · from integers to vectors

Token 4271 is not four thousand of anything. The integer is a name, and arithmetic on names is meaningless.

The obvious fix is one-hot: 20,000 columns, one of them 1. That is a 20,000-dimensional vector per *token*, and 256 of them per review.

> **THE EMBEDDING IS THE FIX**
>
> A learned table with one row per token and d columns. Looking up row i is exactly multiplying the one-hot vector by that table — done without ever forming the one-hot vector.

An embedding is a parameter, not a preprocessing step

```python
import torch.nn as nn

emb = nn.Embedding(num_embeddings=20_000, embedding_dim=128,
                   padding_idx=0)

x = torch.randint(0, 20_000, (32, 256))   # a batch of token ids
print(emb(x).shape)                       # torch.Size([32, 256, 128])
```

- The table is 20,000 × 128 numbers, and every one of them has a gradient
- `padding_idx=0` pins row 0 at zero and keeps it there — padding must not learn anything
- Two tokens that play the same role end up with similar rows, because the loss has no reason to separate them

Step 3 · the classifier

```
class GRUClassifier(nn.Module):
    def __init__(self, vocab):
        super().__init__()
        self.emb  = nn.Embedding(vocab, 128, padding_idx=0)
        self.rnn  = nn.GRU(128, 64, batch_first=True, bidirectional=True)
        self.head = nn.Linear(2 * 64, 2)

    def forward(self, x, lengths):
        e = self.emb(x)
        packed = nn.utils.rnn.pack_padded_sequence(
            e, lengths.cpu(), batch_first=True, enforce_sorted=False)
        _, h = self.rnn(packed)
        return self.head(torch.cat([h[0], h[1]], dim=1))
```

Thirteen lines, and eleven of them are Lecture 10. The two new ones are the embedding and the packing.

Three choices in that model

Choice	Why
`bidirectional=True`	a review's verdict can be in the first sentence or the last
the final hidden state	one fixed-size summary of a variable-length input
GRU, not LSTM	fewer gates, fewer parameters, and Lecture 16 measured no difference here

`h[0]` is the forward direction's last state, `h[1]` the backward direction's — which, read as a sequence, is the *first* token. Concatenating them gives the head both ends.

`pack_padded_sequence`, and what it prevents

It hands the recurrent layer only the real timesteps, so the state returned for a review of 40 words is the state after its 40th word — not after 216 zeros.

> **TWO THINGS IT BUYS**
>
> Correctness, because the summary is of the review rather than of the padding. And speed, because the layer stops early on short sequences instead of stepping through the whole rectangle.

This slide exists because the version without it also runs, also trains, and also reports a plausible number. We are going to measure how plausible.

The training loop is unchanged

```
1  opt   = torch.optim.Adam(model.parameters(), lr=1e-3)
2  lossf = nn.CrossEntropyLoss()
3
4  for epoch in range(4):
5      model.train()
6      for xb, lb, yb in batches:
7          opt.zero_grad()
8          out  = model(xb, lb)
9          loss = lossf(out, yb)
10         loss.backward()
11         opt.step()
```

Exactly the five lines from Lecture 10, in the same order, with the same two silent failures waiting if any of them is deleted. Nothing about text changes the loop.

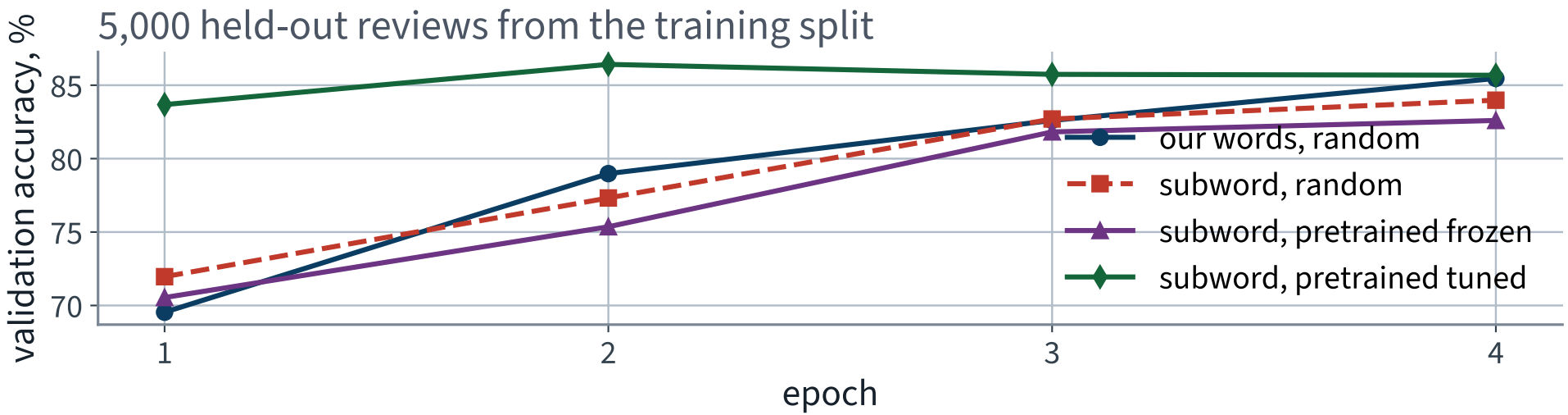
Note line 2. What `CrossEntropyLoss` is actually given, and why, is the whole mathematical thread of the next lecture.

What this costs to run

Setting	Value
Reviews used to fit	20,000
Reviews held out for validation	5,000
Reviews in the test set, untouched	25,000
Epochs, batch size, learning rate	4, 64, 0.001
Parameters	2,634,754
✗ **Wall clock, one run**	✗ **460 s**

Measured on an Apple MPS backend. A Colab T4 is faster; a CPU is several times slower, which is why the device slide came first.

Four epochs



Ignore three of the four curves for now. The blue one is the model we have just built.

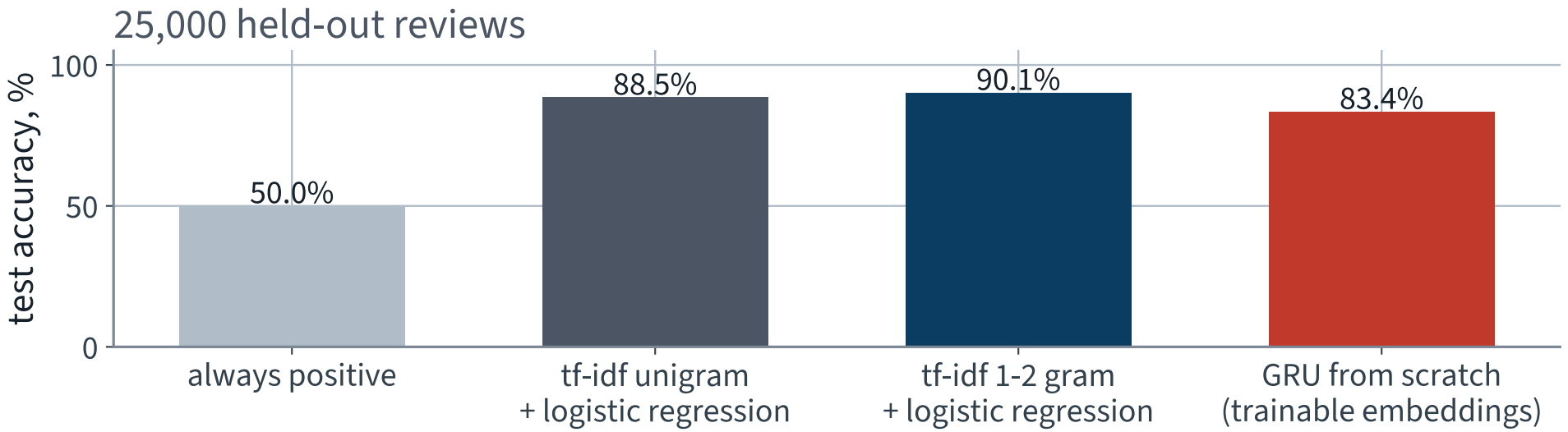
The result

Model	Test accuracy
Always one class	50.0%
tf-idf + logistic regression	90.1%
GRU, our tokenizer, trainable embeddings	83.4%

Write the third number on your sheet.

Then look at the second one for a moment before turning the page.

Against the anchors



✗ The recurrent network, with an embedding it learned itself, loses to counting words.

Why it loses

- The bag of words starts with 200,000 features that are already meaningful. *Terrible* is its own column
- Our network starts with 20,000 rows of **random numbers** and has to discover from 20,000 labelled reviews that *terrible* and *awful* are related
- Two words in five occur once. One example is not enough to learn a 128-dimensional vector

> **THE HYPOTHESIS — AND IT IS ONLY A HYPOTHESIS**
>
> The architecture is not the bottleneck. The **embedding table** is, and it is being asked to learn the English language from 20,000 sentiment labels.

Written as a hypothesis on purpose. The next twenty minutes are an experiment that can refute it, and you should decide now what result would.

THE LAST FIFTEEN MINUTES

The same architecture, better vectors

15 minutes

The diagnosis said the embedding table

So replace the table. Nothing else.

- Same GRU, same width, same head, same optimiser, same learning rate
- Same four epochs, same seed, same 20,000 reviews
- Only the tokenizer and the initial contents of `emb.weight` change

Two changes, applied one at a time, so that the measurement says which one did the work. Changing both at once and reporting the total is how you learn nothing from an experiment.

Swap 1 · the pretrained tokenizer alone

```
from transformers import AutoTokenizer

tk = AutoTokenizer.from_pretrained("distilbert-base-uncased")
enc = tk(texts, truncation=True, max_length=256, padding="max_length")
model = GRUClassifier(vocab=tk.vocab_size)      # 30,522 rows, random
```

Subword pieces instead of words. 30,522 rows instead of 20,000. **Random initial values, exactly as before.**

Predict what this does before you turn the page. Most rooms predict it wrong.

Swap 1, measured

Tokenizer	Vocabulary	Test accuracy
our words	20,000	83.4%
subword pieces, random vectors	30,522	82.5%

No better.

-0.94 accuracy points, from one seed each — which is not enough to call a ranking, and we will not. What the measurement does say is that solving the out-of-vocabulary problem, on its own, **bought nothing**.

Swap 2 · and now the vectors

The same subword vocabulary has an embedding table that was already trained — on far more text than this corpus, by someone else, for a different task.

```
from transformers import AutoModel

bert = AutoModel.from_pretrained("distilbert-base-uncased")
E = bert.embeddings.word_embeddings.weight.detach().numpy()
print(E.shape)          # (30522, 768)
```

✗ 768 columns; our architecture has 128. Widening the GRU would change the architecture, and then the comparison measures two things at once.

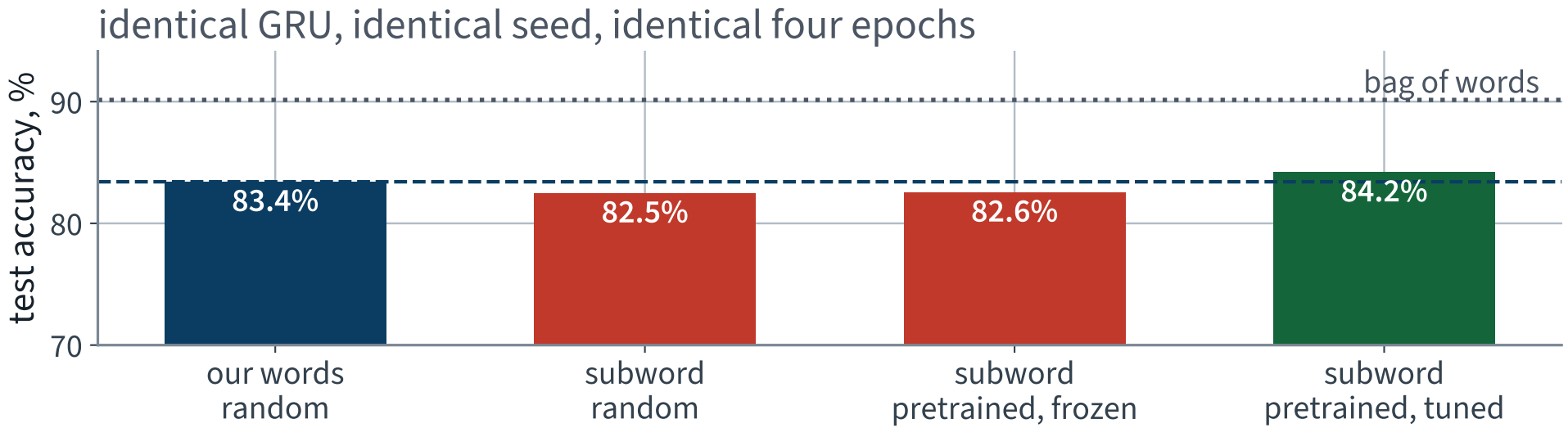
Then one more decision

```
model.emb.weight.data.copy_(torch.from_numpy(Z))
model.emb.weight.requires_grad = False       # frozen  — or True, tuned
```

	Frozen	Fine-tuned
What is learned	the GRU and head only	everything
Risk	the vectors may not suit the task	20,000 labels can overwrite them

Which is better is not decidable from the armchair. Run both; it is one boolean.

Four runs, one architecture



Two things differ across these four bars: which tokenizer produced the ids, and what was in the embedding table at step zero.

The swap, measured

Tokenizer	Embedding table starts as	Test accuracy
our words	random	83.4%
subword	random	82.5%
subword	pretrained, frozen	82.6%
✓ subword	**✓ pretrained, tuned**	**84.2% ✓**

0.79 points, and not one line of the model changed.

Positive, and **small** — about the size of the swing we refused to call a ranking two slides ago. Only the last row beats what you built this morning, and only by about a point.

What moved, and what did not

Unchanged	Changed
GRU width, depth, direction	the tokenizer
the head, the loss, the optimiser	the initial values in one tensor
epochs, batch size, learning rate, seed	whether that tensor is trainable
the data, the split, the metric	

Everything on the left is what you would ordinarily spend the afternoon tuning. Everything on the right took ten minutes — and on this corpus it was worth 0.79 points, which is the honest size of the effect rather than the one the phrase “transfer learning” suggests.

And a warning about that validation set

> **FROM THE CORPUS README, NOT FROM US**
>
> “the train and test sets contain a disjoint set of movies, so no significant performance is obtained by memorizing movie-unique terms”

Our validation split was carved out of the *training* half, so it shares films with the fit split — and a model that memorises film-specific vocabulary is flattered by it. The test half shares no films with anything, which is why the drop from validation to test is larger for the run that overfits most.

✗ A validation set drawn from the same pool as the training data answers “when should I stop?” It does not answer “how well will this do?”

And against the anchor?

Model	Test accuracy
Always one class	50.0%
tf-idf + logistic regression	90.1%
GRU, our tokenizer, random embeddings	83.4%
✓ GRU, subword, pretrained embeddings	**84.2% ✓**

Read the last two rows against the second. That is the question to take home, and the sheet in front of you has a line for it.

Where we are

Model	Test accuracy	To fit	To score 25,000
Always one class	50.0%	—	—
tf-idf + logistic regression	90.1%	10 s	**3.6 s ✓**
GRU from scratch	83.4%	460 s	6.3 s
✓ GRU, pretrained embeddings	**84.2% ✓**	302 s	12.0 s
✗ the assistant's version	**✗ 65.2%**	173 s	6.3 s

Two clocks, and they are not the same clock. The one that matters to the desk is the last column, because it is paid once per review forever; the middle one is paid once. The best accuracy here is still 6.72 points behind counting words, at nearly twice the cost to run.

Text checklist — keep this

- Was the vocabulary built on the training split only?
- Is the tokenizer the same object at fit time and at inference?
- Are true lengths carried alongside the padded batch?
- Does padding change the output? Assert that it does not
- What fraction of the test tokens is `[UNK]`? Count it
- How much of each review survives truncation? Count that too

Six questions. The first is the one that will cost you the most, and it is the one the next lecture ends on.

What we did

1. Turned symbols into vectors: subword tokenisation, so that an unseen word is still representable, and a trainable embedding table
2. Derived softmax and its invariance under a constant shift — which is what makes the max trick both safe and necessary
3. Derived the gradient of cross-entropy with respect to the logits, and found it is exactly **prediction minus target**
4. Saw why the loss consumes logits rather than probabilities: the cancellation is what keeps it stable, and doing the softmax yourself throws it away
5. Related cross-entropy to KL divergence, and measured that they differ by a constant
6. Built a recurrent classifier, then replaced its embeddings with pretrained ones and measured what moved

NOT PRESENTED — FOR AFTERWARDS

Exercises Lecture 17

five, in the style of the written paper

Exercises — Lecture 17

1 Show that softmax is invariant to adding a constant to every logit, and state the numerical reason the implementation subtracts the maximum anyway. [5]

2 Derive $\partial L / \partial \mathbf{z} = \mathbf{p} - \mathbf{y}$ for cross-entropy on a one-hot target, and state what the row sum of that gradient must be. [5]

3 A loss function is given probabilities rather than logits. State the two failure modes, and which one ships. [5]

Solutions on Lecture 18's deck.

Exercises — Lecture 17 (continued)

4 A model summarises a padded batch at `out[:, -1, :]`. State what that position holds for most reviews, and the invariance test that detects it. [5]

5 Enlarging a word vocabulary drives the share of unseen distinct test words down, but the curve flattens above 40% and never reaches zero. Explain why a larger vocabulary is not the fix, and name what is. [4]

Solutions on Lecture 18’s deck.

THE FIVE SET AT THE END OF LECTURE 16

Solutions Lecture 16

not part of this lecture

Solutions — Lecture 16

1. A recurrent network is trained on 56-day windows drawn from one series. Explain why shuffling the windows is…

✓ A window is a training example; the series is the thing the example is cut from.

- The order inside a window is the signal, and shuffling windows leaves it intact
- Shuffling the series destroys the very structure the model exists to read

2. The head of the recurrent model is a linear layer on the final hidden state. State why the hidden state alone…

✓ tanh bounds it to $(-1, 1)$ and ridership is not bounded.

- A bounded output cannot reach the target range
- Scaling by a million makes it half-work, which is worse than failing

Solutions — Lecture 16 (2)

3. Gates were introduced after a plain recurrent cell failed over long windows. State the mechanism, in terms of…

✓ The same matrix is applied 56 times, so its spectrum is raised to the 56th power; gates provide a path whose derivative is near one.

- Repeated multiplication is the vanishing-gradient argument of Lecture 11 in a new place
- A gate lets information pass without being multiplied at every step

4. A colleague reports a large improvement on the ridership forecast after making the recurrent layer…

✓ It is leakage: a bidirectional layer reads the future. Ask whether the whole sequence is available at prediction time.

- A second layer run backwards lets every position see values that have not happened when the forecast is made
- It is Lecture 15's random split wearing a different hat — the number goes up and the system cannot be deployed

Solutions — Lecture 16 (3)

5. Report the training MAE beside the test MAE. Name the two failures the pair distinguishes, and say why the…

✓ Underfitting, where both are poor, and overfitting, where train is far better. They have opposite fixes.

- A single bad test number is consistent with either
- More capacity fixes one and makes the other worse

LECTURE 18 · GÉRON CH. 14–15

Attention and transformers

Scaled dot-product attention, derived — and why the scores are divided by $\sqrt{d_k}$
Multi-head attention, and why a permutation-invariant model needs positions
Fine-tuning a pretrained transformer against today's model

Run the Lecture 17 notebook first